

Workshop 4: Containerization, Acceptance Testing, and CI/CD

Software Engineering Seminar
Semester 2025-III

Team Members:

Carlos Andres Abella

Daniel Felipe Paez

Leidy Marcela Morales

Supervisor: Eng. Carlos Andrés Sierra, M.Sc.

29 de noviembre de 2025

Índice

Índice de figuras

Índice de cuadros

Introduction

This document presents the solution for Workshop 4 of the Software Engineering Seminar course. The workshop focuses on deploying and validating the Event Platform application using containerization, acceptance testing, and CI/CD practices. The goal is to ensure the project is ready for production-like environments and meets quality standards.

The Event Platform is a distributed system composed of three main components:

- **Java Backend:** Spring Boot application handling authentication and user management (Port 8081)
- **Python Backend:** FastAPI application managing events, tickets, and orders (Port 8000)
- **React Frontend:** User interface for ticket buyers and event organizers (Port 3000)

The platform also utilizes two database systems:

- **MySQL:** Used by the Java backend for authentication data
- **PostgreSQL:** Used by the Python backend for business logic data

Docker Containerization

This section describes the containerization strategy implemented for all components of the Event Platform using Docker and Docker Compose.

Overview

Docker containerization provides a consistent environment across development, testing, and production. Each component of the application has been containerized with optimized multi-stage builds where applicable, reducing image size and improving security.

Java Backend Dockerfile

The Java backend uses a multi-stage build to separate the build environment from the runtime environment. This approach significantly reduces the final image size by only including the compiled JAR file and JRE in the runtime stage.

```
1 # Java Backend Dockerfile
2 FROM maven:3.9-eclipse-temurin-17 AS build
3
4 WORKDIR /app
5
6 # Copy pom.xml first for dependency caching
7 COPY pom.xml .
8 RUN mvn dependency:go-offline -B
9
10 # Copy source code
11 COPY src ./src
12
13 # Build the application
14 RUN mvn clean package -DskipTests
```

```
15
16 # Runtime stage
17 FROM eclipse-temurin:17-jre-alpine
18
19 WORKDIR /app
20
21 # Copy the built JAR from build stage
22 COPY --from=build /app/target/*.jar app.jar
23
24 # Expose port
25 EXPOSE 8081
26
27 # Health check
28 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
29     CMD wget --no-verbose --tries=1 --spider http://localhost:8081/health
30     || exit 1
31
32 # Run the application
33 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 1: Java Backend Dockerfile

Key Features:

- Multi-stage build reduces final image size
- Dependency caching by copying `pom.xml` first
- Alpine-based runtime image for minimal size
- Health check endpoint configured for container orchestration
- Port 8081 exposed for external access

Python Backend Dockerfile

The Python backend Dockerfile uses Python 3.12-slim as the base image and installs necessary system dependencies for PostgreSQL connectivity.

```
1 # Python Backend Dockerfile
2 FROM python:3.12-slim
3
4 WORKDIR /app
5
6 # Install system dependencies
7 RUN apt-get update && apt-get install -y \
8     gcc \
9     postgresql-client \
10    && rm -rf /var/lib/apt/lists/*
11
12 # Copy requirements file
13 COPY requirements.txt .
14
15 # Install Python dependencies
16 RUN pip install --no-cache-dir -r requirements.txt
17
18 # Copy application code
19 COPY . .
```

```
20
21 # Expose port
22 EXPOSE 8000
23
24 # Health check
25 HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
26   CMD python -c "import urllib.request; urllib.request.urlopen('http://
    localhost:8000/health')" || exit 1
27
28 # Run the application
29 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Listing 2: Python Backend Dockerfile

Key Features:

- Slim Python image for reduced size
- PostgreSQL client included for database operations
- Health check using Python's urllib module
- Port 8000 exposed for API access

React Frontend Dockerfile

The React frontend uses a multi-stage build: first building the application with Node.js, then serving it with Nginx in production mode.

```
1 # Multi-stage build for React Frontend
2 FROM node:20-alpine AS build
3
4 WORKDIR /app
5
6 # Copy package files
7 COPY package*.json ./
8
9 # Install dependencies
10 RUN npm install
11
12 # Copy source code
13 COPY . .
14
15 # Build the application
16 RUN npm run build
17
18 # Production stage with Nginx
19 FROM nginx:alpine
20
21 # Copy built files from build stage
22 COPY --from=build /app/build /usr/share/nginx/html
23
24 # Copy nginx configuration
25 COPY nginx.conf /etc/nginx/conf.d/default.conf
26
27 # Expose port
28 EXPOSE 80
29
```

```

30 # Health check
31 HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
32   CMD wget --no-verbose --tries=1 --spider http://localhost/ || exit 1
33
34 # Start nginx
35 CMD ["nginx", "-g", "daemon off;"]

```

Listing 3: React Frontend Dockerfile

Key Features:

- Multi-stage build separates build tools from production runtime
- Alpine-based images for minimal footprint
- Nginx serves static files efficiently
- Custom Nginx configuration for SPA routing
- Port 80 exposed (mapped to 3000 in docker-compose)

Docker Compose Configuration

Docker Compose orchestrates all services, including databases, ensuring proper service dependencies and network isolation.

```

1 version: '3.8'
2
3 services:
4   # MySQL Database for Java Backend
5   mysql:
6     image: mysql:8.0
7     container_name: eventplatform-mysql
8     environment:
9       MYSQL_ROOT_PASSWORD: rootpassword
10      MYSQL_DATABASE: eventplatform_auth
11      MYSQL_USER: app_user
12      MYSQL_PASSWORD: AppStrongPass1!
13     ports:
14       - "3307:3306"
15     volumes:
16       - mysql_data:/var/lib/mysql
17       - ./java-backend/scripts/01-create-database.sql:/docker-entrypoint
18         -initdb.d/01-create-database.sql:ro
19       - ./java-backend/scripts/02-seed-data.sql:/docker-entrypoint-
20         initdb.d/02-seed-data.sql:ro
21     healthcheck:
22       test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root",
23         "-prootpassword"]
24       interval: 10s
25       timeout: 5s
26       retries: 5
27     networks:
28       - eventplatform-network
29
30 # PostgreSQL Database for Python Backend
31 postgres:
32   image: postgres:15-alpine

```



```

30     container_name: eventplatform-postgres
31     environment:
32         POSTGRES_USER: postgres
33         POSTGRES_PASSWORD: postgres
34         POSTGRES_DB: eventplatform
35     ports:
36         - "5433:5432"
37     volumes:
38         - postgres_data:/var/lib/postgresql/data
39         - ./python-backend/scripts/01-create-database.sql:/docker-
entrypoint-initdb.d/01-create-database.sql:ro
40         - ./python-backend/scripts/02-setup-schema-and-data.sql:/docker-
entrypoint-initdb.d/02-setup-schema-and-data.sql:ro
41     healthcheck:
42         test: ["CMD-SHELL", "pg_isready -U postgres"]
43         interval: 10s
44         timeout: 5s
45         retries: 5
46     networks:
47         - eventplatform-network
48
49 # Java Backend (Spring Boot)
50 java-backend:
51     build:
52         context: ./java-backend
53         dockerfile: Dockerfile
54     container_name: eventplatform-java-backend
55     environment:
56         SPRING_PROFILES_ACTIVE: docker
57         SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/eventplatform_auth?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
58         SPRING_DATASOURCE_USERNAME: app_user
59         SPRING_DATASOURCE_PASSWORD: AppStrongPass1!
60         JWT_SECRET: your-256-bit-secret-key-change-this-in-production-make
-it-very-long-and-secure
61         JWT_EXPIRATION: 86400000
62         CORS_ALLOWED_ORIGINS: http://localhost:3000,http://localhost:80
63         SERVER_PORT: 8081
64     ports:
65         - "8081:8081"
66     depends_on:
67         mysql:
68             condition: service_healthy
69     networks:
70         - eventplatform-network
71     restart: unless-stopped
72
73 # Python Backend (FastAPI)
74 python-backend:
75     build:
76         context: ./python-backend
77         dockerfile: Dockerfile
78     container_name: eventplatform-python-backend
79     environment:
80         POSTGRES_HOST: postgres
81         POSTGRES_USER: postgres
82         POSTGRES_PASSWORD: postgres
83         POSTGRES_DB: eventplatform

```

```
84     DATABASE_URL: postgresql://postgres:postgres@postgres:5432/
eventplatform
85     JWT_SECRET: your-256-bit-secret-key-change-this-in-production-make
-it-very-long-and-secure
86     JWT_ALGORITHM: HS256
87     CORS_ORIGINS: http://localhost:3000,http://localhost:80
88     ports:
89     - "8000:8000"
90     depends_on:
91     postgres:
92         condition: service_healthy
93     networks:
94     - eventplatform-network
95     restart: unless-stopped
96
97 # React Frontend (Nginx)
98 react-frontend:
99     build:
100         context: ./react-frontend
101         dockerfile: Dockerfile
102         container_name: eventplatform-react-frontend
103     ports:
104     - "3000:80"
105     depends_on:
106     - java-backend
107     - python-backend
108     networks:
109     - eventplatform-network
110     restart: unless-stopped
111
112 volumes:
113     mysql_data:
114         driver: local
115     postgres_data:
116         driver: local
117
118 networks:
119     eventplatform-network:
120         driver: bridge
```

Listing 4: docker-compose.yml

Orchestration Features:

- **Service Dependencies:** Backend services wait for database health checks
- **Network Isolation:** All services communicate through a dedicated bridge network
- **Volume Persistence:** Database data persists across container restarts
- **Health Checks:** Databases include health checks to ensure readiness
- **Automatic Restart:** Services automatically restart on failure
- **Port Mapping:** External ports mapped to avoid conflicts (3307, 5433)

Containerization Benefits

The containerization approach provides several advantages:

1. **Consistency:** Same environment across all stages of development
2. **Isolation:** Services are isolated and cannot interfere with each other
3. **Scalability:** Easy to scale individual services independently
4. **Portability:** Application runs identically on any Docker-compatible system
5. **Resource Efficiency:** Containers share the host OS kernel
6. **Easy Deployment:** Single command deploys entire application stack

Acceptance Testing with Apache Cucumber

Acceptance testing validates that the system meets user requirements and business expectations. Apache Cucumber enables behavior-driven development (BDD) by allowing tests to be written in natural language.

Overview

Cucumber tests are written in Gherkin syntax, which is human-readable and can be understood by both technical and non-technical stakeholders. These tests serve as living documentation of the system's behavior.

Cucumber Feature Files

Feature files describe user stories and scenarios using the Given-When-Then format. The authentication feature covers the core user registration and login functionality.

```
1 Feature: User authentication
2   As a visitor of the event platform
3   I want to be able to register and log in
4   So that I can access the application as a ticket buyer
5
6   Scenario: Successful registration of a BUYER
7     Given a registration request with name "Test User", email "
8     test@example.com", password "Test123!@", user type "BUYER"
9     When the client sends a POST request to "/api/auth/register"
10    Then the response status should be 201
11    And the JSON response should have field "email" equal to "
12    test@example.com"
13    And the JSON response should have field "role" equal to "ROLE_BUYER"
14
15  Scenario: Successful login of a registered user
16    Given an existing user with email "test@example.com" and password "
17    Test123!@"
18    When the client sends a POST request to "/api/auth/login"
19    Then the response status should be 200
20    And the JSON response should have field "email" equal to "
21    test@example.com"
```

```
18 And the JSON response should have field "token" present
```

Listing 5: Authentication Feature File

Step Definitions

Step definitions implement the behavior described in feature files. The `AuthStepDefinitions` class maps Gherkin steps to Java code using Spring's `MockMvc` for integration testing.

```
1 package com.eventplatform.acceptance;
2
3 import com.eventplatform.dto.LoginRequest;
4 import com.eventplatform.dto.RegisterRequest;
5 import com.fasterxml.jackson.databind.ObjectMapper;
6 import io.cucumber.java.en.And;
7 import io.cucumber.java.en.Given;
8 import io.cucumber.java.en.Then;
9 import io.cucumber.java.en.When;
10 import org.springframework.beans.factory.annotation.Autowired;
11 import org.springframework.http.MediaType;
12 import org.springframework.test.web.servlet.MockMvc;
13 import org.springframework.test.web.servlet.ResultActions;
14
15 import static org.hamcrest.Matchers.notNullValue;
16 import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.post;
17 import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.jsonPath;
18 import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.status;
19
20 public class AuthStepDefinitions {
21
22     @Autowired
23     private MockMvc mockMvc;
24
25     @Autowired
26     private ObjectMapper objectMapper;
27
28     private RegisterRequest registerRequest;
29     private LoginRequest loginRequest;
30     private ResultActions lastResponse;
31
32     @Given("a registration request with name {string}, email {string},
    password {string}, user type {string}")
33     public void aRegistrationRequest(String name, String email, String
    password, String userType) {
34         registerRequest = new RegisterRequest();
35         registerRequest.setName(name);
36         registerRequest.setEmail(email);
37         registerRequest.setPassword(password);
38         registerRequest.setUserType(userType);
39     }
40
41     @When("the client sends a POST request to {string}")
42     public void theClientSendsPost(String path) throws Exception {
43         Object body;
```

```

44     if (path.contains("/register")) {
45         body = registerRequest;
46     } else {
47         body = loginRequest;
48     }
49
50     lastResponse = mockMvc.perform(
51         post(path)
52             .contentType(MediaType.APPLICATION_JSON)
53             .content(objectMapper.writeValueAsString(body))
54     );
55 }
56
57 @Then("the response status should be {int}")
58 public void theResponseStatusShouldBe(int status) throws Exception {
59     lastResponse.andExpect(status().is(status));
60 }
61
62 @And("the JSON response should have field {string} equal to {string}")
63 public void theJsonResponseShouldHaveFieldEqualTo(String field,
64     String value) throws Exception {
65     lastResponse.andExpect(jsonPath("$. " + field).value(value));
66 }
67
68 @And("the JSON response should have field {string} present")
69 public void theJsonResponseShouldHaveFieldPresent(String field)
70     throws Exception {
71     lastResponse.andExpect(jsonPath("$. " + field, notNullValue()));
72 }
73
74 @Given("an existing user with email {string} and password {string}")
75 public void anExistingUserWithEmailAndPassword(String email, String
76     password) {
77     loginRequest = new LoginRequest();
78     loginRequest.setEmail(email);
79     loginRequest.setPassword(password);
80 }

```

Listing 6: Auth Step Definitions (Partial)

Cucumber Test Runner Configuration

The test runner configuration integrates Cucumber with Spring Boot's test framework, enabling dependency injection and Spring context loading.

```

1 package com.eventplatform.acceptance;
2
3 import io.cucumber.spring.CucumberContextConfiguration;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.test.context.ActiveProfiles;
6
7 @CucumberContextConfiguration
8 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
9 @ActiveProfiles("test")
10 public class CucumberSpringConfiguration {

```

11 }

Listing 7: Cucumber Spring Configuration

Test Execution and Results

Cucumber tests are executed using Maven's test phase. The test runner generates both console output and HTML reports showing test results. Successful execution indicates that:

- User registration creates accounts with correct roles
- Login authentication generates valid JWT tokens
- API responses match expected status codes
- JSON responses contain required fields

Benefits of Acceptance Testing

1. **Documentation:** Feature files serve as executable specifications
2. **Communication:** Business stakeholders can read and understand tests
3. **Regression Prevention:** Tests catch breaking changes early
4. **User-Centric:** Tests focus on user stories and business value
5. **Integration Validation:** Tests verify end-to-end functionality

API Stress Testing with Apache JMeter

Esta sección describe las pruebas de estrés realizadas sobre las APIs REST utilizando Apache JMeter para evaluar el rendimiento y la capacidad del sistema bajo carga.

Introducción

Las pruebas de estrés son fundamentales para determinar los límites del sistema y asegurar que la plataforma pueda manejar cargas de trabajo esperadas en producción. Apache JMeter permite simular múltiples usuarios concurrentes y medir métricas de rendimiento como tiempo de respuesta, throughput y tasa de error.

Plan de Pruebas

Se diseñaron planes de prueba para evaluar diferentes endpoints críticos:

- **Autenticación:** Login y registro de usuarios
- **Eventos:** Consulta y creación de eventos
- **Tickets:** Compra y consulta de tickets
- **Órdenes:** Procesamiento de órdenes de compra

Configuración del Test Plan

Cada plan de prueba incluye:

1. **Thread Group:** Configuración de usuarios virtuales (número, tiempo de ramp-up, iteraciones)
2. **HTTP Request Samplers:** Endpoints a probar con métodos HTTP correspondientes
3. **Headers:** Configuración de Content-Type, Authorization cuando aplica
4. **Listeners:** Recolección de resultados (View Results Tree, Summary Report, Graph Results)

Escenarios de Prueba

4.4.1. Escenario 1: Carga Gradual en Login

Este escenario simula un aumento gradual de usuarios realizando login simultáneamente.

Configuración:

- Número de usuarios: 100
- Ramp-up period: 60 segundos
- Duración: 5 minutos
- Endpoint: POST /api/auth/login

4.4.2. Escenario 2: Pico de Carga en Consulta de Eventos

Simula un pico de tráfico cuando muchos usuarios consultan eventos simultáneamente.

Configuración:

- Número de usuarios: 200
- Ramp-up period: 10 segundos
- Duración: 3 minutos
- Endpoint: GET /api/events

4.4.3. Escenario 3: Carga Sostenida en Compra de Tickets

Evalúa el comportamiento del sistema durante operaciones de compra bajo carga sostenida.

Configuración:

- Número de usuarios: 50
- Ramp-up period: 30 segundos
- Duración: 10 minutos
- Endpoint: POST /api/orders

Métricas Evaluadas

Las pruebas miden las siguientes métricas clave:

- **Tiempo de Respuesta (Response Time):** Tiempo promedio, mediana, mínimo y máximo
- **Throughput:** Peticiones por segundo que el servidor puede manejar
- **Tasa de Error (Error Rate):** Porcentaje de peticiones fallidas
- **Latencia:** Tiempo desde la petición hasta la primera respuesta
- **Utilización de Recursos:** Uso de CPU y memoria (si se monitorea el servidor)

Resultados Esperados

Basado en los resultados de las pruebas de estrés, se espera que:

1. El sistema maneje al menos 50 usuarios concurrentes sin degradación significativa
2. El tiempo de respuesta promedio sea inferior a 500ms para endpoints de consulta
3. El tiempo de respuesta para operaciones de escritura (POST) sea inferior a 1 segundo
4. La tasa de error sea inferior al 1 % bajo carga normal
5. El throughput sea al menos 100 peticiones por segundo para endpoints de consulta

Análisis de Resultados

Los resultados de las pruebas de estrés permiten identificar:

- **Cuellos de botella:** Endpoints que requieren optimización
- **Límites de capacidad:** Número máximo de usuarios concurrentes
- **Problemas de escalabilidad:** Comportamiento del sistema bajo diferentes cargas
- **Problemas de concurrencia:** Race conditions o bloqueos en la base de datos

Recomendaciones

Basado en los resultados, se recomienda:

1. Implementar caché para endpoints de consulta frecuente
2. Optimizar consultas a base de datos mediante índices apropiados
3. Considerar implementar rate limiting para prevenir abuso
4. Monitorear métricas de rendimiento en producción
5. Escalar horizontalmente cuando la carga lo requiera

CI/CD Pipeline with GitHub Actions

Continuous Integration and Continuous Deployment (CI/CD) automates the testing and building processes, ensuring code quality and enabling rapid, reliable deployments.

Overview

The CI/CD pipeline implemented using GitHub Actions automates:

- Running tests on every push and pull request
- Building Docker images for all components
- Validating code quality and security
- Preparing artifacts for deployment

GitHub Actions Workflow Configuration

The workflow is triggered on push and pull request events to main and develop branches. It runs tests and builds Docker images in parallel for efficient execution.

```
1 name: CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ main, develop ]
6   pull_request:
7     branches: [ main, develop ]
8
9 env:
10  JAVA_VERSION: '17'
11  PYTHON_VERSION: '3.12'
12  NODE_VERSION: '20'
13
14 jobs:
15   test-java-backend:
16     name: Test Java Backend
17     runs-on: ubuntu-latest
18     services:
19       mysql:
20         image: mysql:8.0
21         env:
22           MYSQL_ROOT_PASSWORD: rootpassword
23           MYSQL_DATABASE: eventplatform_auth
24         ports:
25           - 3306:3306
26         options: >-
27           --health-cmd="mysqladmin ping"
28           --health-interval=10s
29           --health-timeout=5s
30           --health-retries=5
31
32   steps:
33     - name: Checkout code
34       uses: actions/checkout@v3
```

```

35
36   - name: Set up JDK ${ env.JAVA_VERSION }}
37     uses: actions/setup-java@v3
38     with:
39       java-version: ${ env.JAVA_VERSION }}
40       distribution: 'temurin'
41       cache: 'maven'
42
43   - name: Run unit tests
44     working-directory: ./Workshop-3/java-backend
45     run: mvn clean test
46
47   - name: Run acceptance tests
48     working-directory: ./Workshop-3/tests/java-backend
49     run: mvn clean test
50
51   - name: Publish test results
52     uses: EnricoMi/publish-unit-test-result-action@v2
53     if: always()
54     with:
55       files: |
56         Workshop-3/tests/java-backend/target/surefire-reports/*.xml
57
58 test-python-backend:
59   name: Test Python Backend
60   runs-on: ubuntu-latest
61   services:
62     postgres:
63       image: postgres:15-alpine
64       env:
65         POSTGRES_USER: postgres
66         POSTGRES_PASSWORD: postgres
67         POSTGRES_DB: eventplatform
68       ports:
69         - 5432:5432
70       options: >-
71         --health-cmd pg_isready
72         --health-interval 10s
73         --health-timeout 5s
74         --health-retries 5
75
76   steps:
77     - name: Checkout code
78       uses: actions/checkout@v3
79
80     - name: Set up Python ${ env.PYTHON_VERSION }}
81       uses: actions/setup-python@v4
82       with:
83         python-version: ${ env.PYTHON_VERSION }}
84         cache: 'pip'
85
86     - name: Install dependencies
87       working-directory: ./Workshop-3/python-backend
88       run: |
89         pip install -r requirements.txt
90         pip install pytest pytest-cov
91
92     - name: Run tests

```

```

93     working-directory: ./Workshop-3/python-backend
94     env:
95       DATABASE_URL: postgresql://postgres:postgres@localhost:5432/
eventplatform
96     run: pytest tests/ -v --cov=app --cov-report=xml
97
98   - name: Upload coverage
99     uses: codecov/codecov-action@v3
100    if: always()
101    with:
102      file: ./Workshop-3/python-backend/coverage.xml
103      flags: python-backend
104
105  build-docker-images:
106    name: Build Docker Images
107    runs-on: ubuntu-latest
108    needs: [test-java-backend, test-python-backend]
109
110    steps:
111      - name: Checkout code
112        uses: actions/checkout@v3
113
114      - name: Set up Docker Buildx
115        uses: docker/setup-buildx-action@v2
116
117      - name: Build Java Backend image
118        uses: docker/build-push-action@v4
119        with:
120          context: ./Workshop-3/java-backend
121          file: ./Workshop-3/java-backend/Dockerfile
122          push: false
123          tags: event-platform/java-backend:latest
124          cache-from: type=registry,ref=event-platform/java-backend:
latest
125            cache-to: type=inline
126
127      - name: Build Python Backend image
128        uses: docker/build-push-action@v4
129        with:
130          context: ./Workshop-3/python-backend
131          file: ./Workshop-3/python-backend/Dockerfile
132          push: false
133          tags: event-platform/python-backend:latest
134          cache-from: type=registry,ref=event-platform/python-backend:
latest
135            cache-to: type=inline
136
137      - name: Build React Frontend image
138        uses: docker/build-push-action@v4
139        with:
140          context: ./Workshop-3/react-frontend
141          file: ./Workshop-3/react-frontend/Dockerfile
142          push: false
143          tags: event-platform/react-frontend:latest
144          cache-from: type=registry,ref=event-platform/react-frontend:
latest
145            cache-to: type=inline
146

```

```

147     - name: Test docker-compose
148       working-directory: ./Workshop-3
149       run: |
150         docker-compose config
151         docker-compose build
152
153 lint-frontend:
154   name: Lint React Frontend
155   runs-on: ubuntu-latest
156
157   steps:
158     - name: Checkout code
159       uses: actions/checkout@v3
160
161     - name: Set up Node.js ${ env.NODE_VERSION }
162       uses: actions/setup-node@v3
163       with:
164         node-version: ${ env.NODE_VERSION }
165         cache: 'npm'
166         cache-dependency-path: Workshop-3/react-frontend/package-lock.
167         json
168
169     - name: Install dependencies
170       working-directory: ./Workshop-3/react-frontend
171       run: npm ci
172
173     - name: Run linter
174       working-directory: ./Workshop-3/react-frontend
175       run: npm run lint || true
176
177     - name: Check build
178       working-directory: ./Workshop-3/react-frontend
179       run: npm run build

```

Listing 8: GitHub Actions CI/CD Workflow

Workflow Stages

The CI/CD pipeline consists of four main stages:

5.3.1. 1. Test Java Backend

- Sets up MySQL service container
- Runs unit tests using Maven
- Executes Cucumber acceptance tests
- Publishes test results

5.3.2. 2. Test Python Backend

- Sets up PostgreSQL service container
- Installs Python dependencies

- Runs pytest with coverage reporting
- Uploads coverage reports to codecov

5.3.3. 3. Build Docker Images

- Runs only after tests pass
- Builds all three Docker images
- Uses Docker layer caching for faster builds
- Validates docker-compose configuration

5.3.4. 4. Lint Frontend

- Installs Node.js dependencies
- Runs ESLint (if configured)
- Validates that the frontend builds successfully

CI/CD Benefits

The implemented CI/CD pipeline provides several advantages:

1. **Early Error Detection:** Tests run automatically on every commit
2. **Consistency:** Same build process across all environments
3. **Faster Feedback:** Developers know immediately if their changes break tests
4. **Automated Quality Gates:** Code must pass tests before merging
5. **Reproducible Builds:** Docker images are built consistently
6. **Time Savings:** Automates repetitive testing and building tasks

Workflow Execution Evidence

The GitHub Actions workflow execution is visible in the repository's Actions tab. Each workflow run shows:

- Execution status (success/failure)
- Duration of each job
- Test results and coverage reports
- Build logs for troubleshooting
- Artifacts generated (if any)

Project Organization and Documentation

This section describes the organization of Workshop 4 deliverables and the documentation structure.

Folder Structure

All Workshop 4 deliverables are organized in the `Workshop-4` folder with the following structure:

```

1 Workshop-4/
2   Workshop-4-Report.tex           # This LaTeX document
3   Workshop-4-Report.pdf          # Compiled PDF
4   README.md                      # Setup and usage guide
5   docker/
6       docker-compose.yml         # Service orchestration
7       java-backend/
8           Dockerfile             # Java backend container
9       python-backend/
10          Dockerfile             # Python backend container
11       react-frontend/
12          Dockerfile             # React frontend container
13   cucumber/
14       features/                  # Feature files
15           authentication.feature
16       step-definitions/          # Step implementation
17       test-results/              # Test execution results
18   jmeter/
19       test-plans/                # JMeter test plans (.jmx)
20       results/                  # Test results and reports
21   ci-cd/
22       .github/
23           workflows/
24               ci-cd.yml           # GitHub Actions workflow

```

Listing 9: Workshop-4 Folder Structure

README.md Structure

The README.md file provides comprehensive setup and usage instructions, including:

- Prerequisites and installation requirements
- Quick start guide with Docker
- Manual setup instructions
- Running tests (Cucumber and JMeter)
- CI/CD pipeline information
- Troubleshooting common issues
- References to relevant documentation

Documentation Standards

All documentation follows these standards:

- Written in English (except JMeter section as specified)
- Clear, concise, and well-structured
- Includes code examples where applicable
- References external resources when used
- Provides step-by-step instructions
- Includes troubleshooting sections

Conclusion

This workshop has successfully implemented containerization, acceptance testing, and CI/CD practices for the Event Platform. The deliverables demonstrate:

1. **Production-Ready Containerization:** All components are containerized with optimized Dockerfiles and orchestrated using Docker Compose
2. **Comprehensive Testing:** Acceptance tests validate user stories using Cucumber, ensuring the system meets business requirements
3. **Performance Validation:** JMeter stress tests evaluate system performance under load
4. **Automated Quality Assurance:** CI/CD pipeline automates testing and building, ensuring code quality on every commit
5. **Professional Documentation:** Well-organized documentation facilitates setup, usage, and maintenance

These practices ensure the Event Platform is ready for deployment to production-like environments and maintainable by development teams. The implemented solutions follow industry best practices and provide a solid foundation for continued development and scaling.

References

1. Docker Documentation. *Dockerfile best practices*. <https://docs.docker.com/develop/dev-best-practices/>
2. Apache Cucumber. *Cucumber Documentation*. <https://cucumber.io/docs/cucumber/>
3. Apache JMeter. *User Manual*. <https://jmeter.apache.org/usermanual/>
4. GitHub Actions Documentation. *Workflow syntax*. <https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions>

5. Spring Boot Testing. *Testing Spring Boot Applications*. <https://spring.io/guides/gs/testing-web/>
6. FastAPI Testing. *Testing FastAPI*. <https://fastapi.tiangolo.com/tutorial/testing/>
7. Docker Compose Documentation. *Overview of Docker Compose*. <https://docs.docker.com/compose/>